

Basic Information

Experiment No.: 07

Date: 08-10-2026

Title: Java PriorityQueue and Its Methods

Objective / Aim

To implement "PriorityQueue" in Java and demonstrate operations such as insertion, accessing, removal, searching, and finding the size of the queue.

System Requirements

Hardware

- Computer/Laptop
- Minimum 4 GB RAM

Software

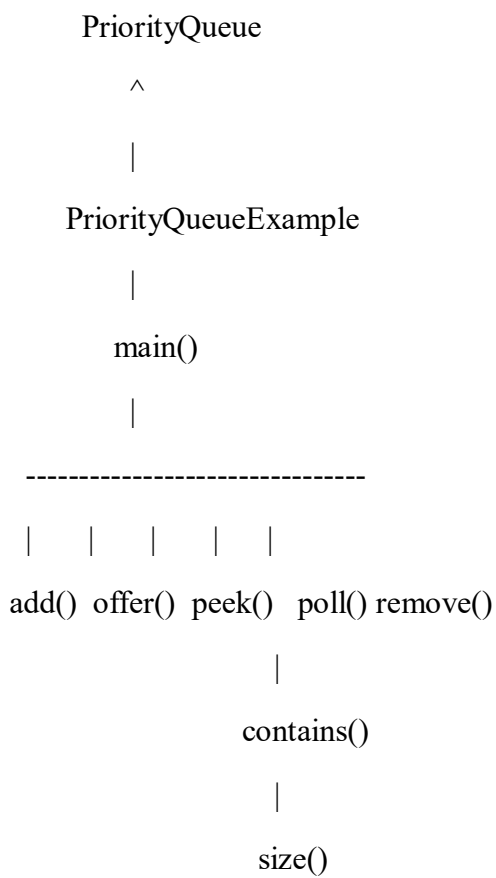
- Java JDK
 - Any Java IDE or Command Prompt
 - Operating System: Windows/Linux/macOS
-

Algorithm / Logic Description

1. Create a "PriorityQueue" of integers.
2. Add elements using "add()" and "offer()".
3. Display the head element using "peek()".
4. Remove the head element using "poll()".
5. Remove a specific element using "remove()".

6. Check an element using "contains()".
 7. Find the number of elements using "size()".
 8. Display the queue after each operation.
 9. End the program.
-

UML Class Diagram



Source Code

```
package collectionframework;

import java.util.PriorityQueue;
```

```
public class PriorityQueueExample {  
    public static void main(String[] args) {  
  
        PriorityQueue<Integer> pq = new PriorityQueue<>();  
  
        // add()  
        pq.add(30);  
        pq.add(10);  
        pq.add(20);  
  
        System.out.println("Priority Queue: " + pq);  
  
        // offer()  
        pq.offer(5);  
        System.out.println("After offer: " + pq);  
  
        // peek()  
        System.out.println("Head: " + pq.peek());  
  
        // poll()  
        System.out.println("Removed Head: " + pq.poll());  
        System.out.println("After poll: " + pq);  
  
        // remove()  
        pq.remove(20);  
        System.out.println("After remove: " + pq);  
  
        // contains()  
        System.out.println("Contains 30: " +
```

```
        pq.contains(30));

    // size()
    System.out.println("Size: " + pq.size());
}
}
```

Compilation & Execution Commands

Save the file as:

PriorityQueueExample.java

Since the program contains:

```
package collectionframework;
```

the file should be inside the "collectionframework" folder.

Compile:

```
javac PriorityQueueExample.java
```

Run:

```
java collectionframework.PriorityQueueExample
```

Output Screen

Priority Queue: [10, 30, 20]

After offer: [5, 10, 20, 30]

Head: 5

Removed Head: 5

After poll: [10, 30, 20]

After remove: [10, 30]

Contains 30: true

Size: 2

Note: "PriorityQueue" maintains priority for the head element, but its complete displayed order is not necessarily sorted.

Result

The "PriorityQueue" class was successfully implemented, and the "add()", "offer()", "peek()", "poll()", "remove()", "contains()", and "size()" operations were successfully demonstrated.